

Lab 9.3 Machine Learning 2 - Machine Learning Implementation to Edge Device

Building Up The Entire Monitoring System

3.1 MTConnect Data Stream

Let's move on to the last step for the entire smart IIoT monitoring system. In this section, we will integrate each monitoring part we practiced in all previous labs into a single system to realize a real-time web-based monitoring system as [Figure 1](#). In the previous section of this lab, we practiced the use of the machine learning model using TensorFlow on Raspberry Pi. In this part, let's make MTConnect data stream including ML results based on the model. The schematic and the data items are shown in [Figure 7](#).

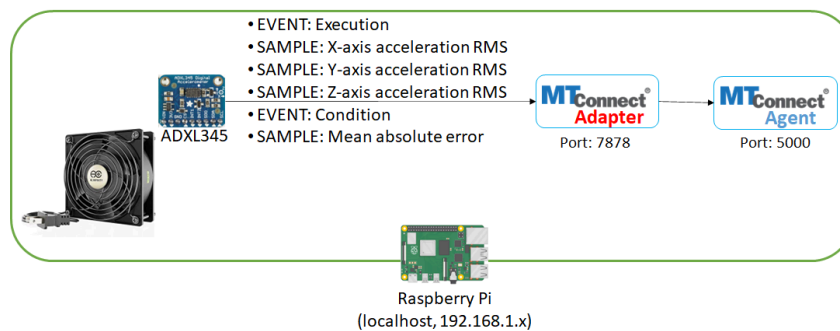


Figure 7 Schematic and data items of MTConnect

The examples of 'agent.cfg' and 'device.xml' are [Figure 8](#) and [Figure 9](#), respectively. You may have different a XML data structure or data item definitions. Also, you can check the example of MTConnect agent running on the server computer: <http://mepotr16.ecn.purdue.edu:5000/>.

```

1 Devices = device.xml
2 Port = 5000
3 AllowPut = true
4 ReconnectInterval = 1000
5 BufferSize = 17
6 MonitorConfigFiles = true
7 Pretty = true
8 SchemaVersion = 1.5
9
10 Adapters {
11   # Log file has all machines with device name prefixed
12   ADXL345 {
13     Host = 127.0.0.1
14     Port = 7878
15   }
16 }

```

Figure 8 front part of 'agent.cfg'

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <MTConnectDevices xmlns:m="urn:mtconnect.org:MTConnectDevices:1.1" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns=
3 "urn:mtconnect.org:MTConnectDevices:1.1" xsi:schemaLocation="urn:mtconnect.org:MTConnectDevices:1.1 http://www.mtconnect.org/schemas/MTConnectDevices_1.1.xsd">
4   <Header creationTime="2010-03-04T18:44:40+00:00" sender="localhost" instanceId="1267728234" bufferSize="131072" version="1.1"/>
5   <Devices>
6     <Device id="adxl345" name="ADXL345" uuid="001">
7       <DataItems>
8         <DataItem category="EVENT" id="001" name="avail" type="AVAILABILITY"/>
9         <DataItem category="EVENT" id="e1" name="execution" type="EXECUTION"/>
10        <DataItem category="EVENT" id="c1" name="condition" type="STATE"/>
11        <DataItem category="SAMPLE" id="error1" name="mae" type="LOSS"/>
12        <DataItem category="SAMPLE" id="a1" name="Xacc" type="ACCELERATION" units="MILLIMETER/SECOND2"/>
13        <DataItem category="SAMPLE" id="a2" name="Yacc" type="ACCELERATION" units="MILLIMETER/SECOND2"/>
14        <DataItem category="SAMPLE" id="a3" name="Zacc" type="ACCELERATION" units="MILLIMETER/SECOND2"/>
15      </DataItems>
16    </Device>
17  </Devices>
</MTConnectDevices>

```

Figure 9 'device.xml'

Task 3.1

Run MTConnect agent on Raspberry Pi with the XML structure and data items defined above.

- Please note that the command to run MTConnect agent is `sudo ./agent` in the same directory of the execution file.
- The '*agent.cfg*' and '*device.xml*' must be in the same directory.
- To apply the schema and style in MTConnect agent, '*/schema*' and '*/styles*' must be in the same directory as well.

Capture '*/current*' response from the MTConnect agent as Figure 10 and attach it to the report.

L9_Colab2.ipynb - Colab

ME-597-Ilo7/lab/lab9/L9_Colab2.ipynb

MTConnect Device Streams

+

Not secure 10.0.0.224:5000/current

Summarize

Chat

Homepage - Purd...Purdue PortalJohn Blair Hooded...Cash & stock | Sto...ASUS E-ServiceOneCampus | Mos...Your Courses | Gra...Other favorites

- creationTime: 2026-04-05T21:10:57Z
- sender: robertclaud
- instanceId: 1775423448
- version: 1.8.0.3
- bufferSize: 131072
- nextSequence: 10
- firstSequence: 1
- lastSequence: 9

Device: ADXL345; UUID: 001

Device : ADXL345

Samples

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-04-05T21:10:48.121794Z	Acceleration		Xacc	a1	2	UNAVAILABLE
2026-04-05T21:10:48.122212Z	Acceleration		Yacc	a2	7	UNAVAILABLE
2026-04-05T21:10:48.121992Z	Acceleration		Zacc	a3	4	UNAVAILABLE
2026-04-05T21:10:48.121906Z	Loss		mae	error1	3	UNAVAILABLE

Events

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-04-05T21:10:48.122372Z	Availability			001	9	UNAVAILABLE
2026-04-05T21:10:48.121106Z	AssetChanged			adxl345_asset_chg	1	UNAVAILABLE
2026-04-05T21:10:48.12215Z	AssetRemoved			adxl345_asset_rem	6	UNAVAILABLE
2026-04-05T21:10:48.12207Z	Condition		condition	c1	5	UNAVAILABLE
2026-04-05T21:10:48.122296Z	Execution		execution	e1	8	UNAVAILABLE

- creationTime: 2022-04-01T03:31:33Z - sender: lamm-UltraPoint - instanceId: 1648783879 - version: 1.8.0.2 - bufferSize: 131072 - nextSequence: 10 - firstSequence: 1 - lastSequence: 9						
Device: ADXL345; UUID: 001						
Device : ADXL345						
Samples						
Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2022-04-01T03:31:19.501339Z	Acceleration		Xacc	a1	2	UNAVAILABLE
2022-04-01T03:31:19.502042Z	Acceleration		Yacc	a2	7	UNAVAILABLE
2022-04-01T03:31:19.501693Z	Acceleration		Zacc	a3	4	UNAVAILABLE
2022-04-01T03:31:19.501559Z	Loss		mae	error1	3	UNAVAILABLE
Events						
Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2022-04-01T03:31:19.502272Z	Availability		avail	001	9	UNAVAILABLE
2022-04-01T03:31:19.500637Z	AssetChanged			adxl345_asset_chg	1	UNAVAILABLE
2022-04-01T03:31:19.501909Z	AssetRemoved			adxl345_asset_rem	6	UNAVAILABLE
2022-04-01T03:31:19.501816Z	State		condition	c1	5	UNAVAILABLE
2022-04-01T03:31:19.502177Z	Execution		execution	e1	8	UNAVAILABLE

Figure 10 MTConnect agent '/current' response without MTConnect adapter

The next is to run MTConnect adapter. The sample MTConnect adapter based on the previous section is prepared, [lab9_adapter_sample.py](#). The sample is incomplete as is. **Please note that to run MTConnect adapter in this case, 'data_item.py' and 'mtconnect_adapter.py' modules as the previous labs. In addition, the '/models' and sub-directory for your autoencoder model must be in the same directory of the MTConnect adapter Python script.** Perform Task 3.2 to finish MTConnect data stream.

Task 3.2

Complete the sample script ([lab9_adapter_sample.py](#)) and then run MTConnect adapter.

- You need to modify the variables wrapped in asteriks below. Also, you need to change the '???' parameters to the appropriate parameters.
- If you have a different XML structure and data items from the example, you must change other parts of the sample script as well.

Python - Python3 ("Main of [lab9_adapter_sample.py](#))

```
if __name__ == "__main__":
    *model_path* = "models/YourModelDirectory/" # model file directory, you must change
    this!
    *model* = tf.keras.models.load_model(model_path) # load the model from the path above
    *threshold* = # float: threshold (MAE loss) for the ML model
    *min_val* = # float: minimum value for normalization
    *max_val* = # float: maximum value for normalization

    # start MTConnect Adapter
    MTConnectAdapter(???, ???) # Args: host ip, port number
```

Python - Python3 (Part of "MTConnectAdapter" class of [lab9_adapter_sample.py](#))

```
def adapter_stream(self):
    while True:
        try:
            x, y, z = measureData(acc,1000) # x=x-axis, y=y-axis, z-axis acceleration
            array
            *input_feature* = # your input feature
```

```

        input_feature_normalized = tensorNormalization(input_feature, ???, ???) #
normalized input feature
        result = predict(model, *FINAL_FEATURE_INPUT*, threshold)
        x_rms = timeFeatures(x)[2] # calculate rms of x-axis
        *y_rms* = # calculate rms of y-axis
        *z_rms* = # calculate rms of z-axis
        mae = result[?] # mean absolute error (loss) of the model

    if *AFF is running*:
        execution = 'ACTIVE'
        if *AFF is normal*:
            condition = 'NORMAL'
        else:
            condition = 'ABNORMAL'
    else:
        execution = 'STOPPED'
        condition = 'UNAVAILABLE'

    now = datetime.datetime.now() # get current data time

    self.adapter.begin_gather() # start to collection

    self.Xacc.set_value(str(???))
    self.Yacc.set_value(str(???))
    self.Zacc.set_value(str(???))
    self.execution.set_value(str(???))
    self.condition.set_value(str(???))
    self.mae.set_value(str(???))

    self.adapter.complete_gather() # end of collection

```

Working code for Task 3.2

```

In [ ]: # MTConnet adapter sample for ME597 Lab10
# Copyright
# Example for Sample and Event

import sys
import time
import datetime
import board
import busio
import adafruit_adxl34x
from micropython import const
import numpy as np
from scipy import stats, fft
import tensorflow as tf
from data_item import Event, Sample # Load data_item package
from mtconnect_adapter import Adapter # Load mtconnect_adapter package
import keras

i2c = busio.I2C(board.SCL, board.SDA) # i2c variable defines I2C interfaces and GPIO pins using busio and bo
acc = adafruit_adxl34x.ADXL345(i2c) # acc object is instantiation using i2c of Adafruit ADXL34X library
acc.data_rate = const(0b1111) # change sampling rate as 3200 Hz

# ratedict=output rate dictionary
# See Table5 of Lab3 manual key=rate code (decimal), value=output data rate (Hz)
ratedict = {15:3200,14:1600,13:800,12:400,11:200,10:100,9:50,8:25,7:12.5,6:6.25,5:3.13,4:1.56,3:0.78,2:0.39,
print("Output data rate is {} Hz".format(ratedict[acc.data_rate])) # printing out data rate

def measureData(sensor:object, N:int): # measuring raw data
    data_x, data_y, data_z = [], [], []
    for i in range(N):

```

```

        x_acc, y_acc, z_acc = sensor.acceleration
        data_x.append(x_acc)
        data_y.append(y_acc)
        data_z.append(z_acc)
    x_data, y_data, z_data = np.array(data_x), np.array(data_y), np.array(data_z)
    return x_data, y_data, z_data

def timeFeatures(data): # time domain signal processing
    mean = np.mean(data) # mean
    std = np.std(data) # standard deviation
    rms = np.sqrt(np.mean(data ** 2)) # root mean square
    peak = np.max(abs(data)) # peak
    skew = stats.skew(data) # skewness
    kurt = stats.kurtosis(data) # kurtosis
    cf = peak/rms # crest factor
    feature = np.array([mean, std, rms, peak, skew, kurt, cf], dtype=float) # number of features is 7
    return feature # numpy array, each element data type = float

def freqFeatures(data): # freq domain signal processing
    N = len(data) # Length of the data (must be 1000)
    yf = 2/N*np.abs(fft.fft(data)[:N//2]) # yf is DFT signal magnitude
    yf[0] = 0
    feature = np.array(yf, dtype=float) # feature array
    return feature # numpy array, each element data type = float

def tensorNormalization(data, min_val, max_val): # data input as numpy array, min, max
    data_normal = (data - min_val) / (max_val - min_val)
    tensor = tf.cast(data_normal, tf.float32)
    tensor_feature = tf.reshape(tensor, [-1, len(tensor)])
    return tensor_feature

def predict(model, data, threshold):
    reconstruction = model(data)
    if isinstance(reconstruction, dict):
        reconstruction = list(reconstruction.values())[0]
    loss = tf.keras.losses.mae(reconstruction, data).numpy()
    result = tf.math.less(loss, threshold).numpy()
    return result[0], loss[0]

class MTConnectAdapter(object): # MTConnect adapter object

    def __init__(self, host, port): # init of MTconnectAdapter class
        # MTConnect adapter connection info
        self.host = host # host arg of adapter
        self.port = port # port arg of adapter
        self.adapter = Adapter((host, port))

        # For samples
        self.Xacc = Sample('Xacc')
        self.adapter.add_data_item(self.Xacc)
        self.Yacc = Sample('Yacc')
        self.adapter.add_data_item(self.Yacc)
        self.Zacc = Sample('Zacc')
        self.adapter.add_data_item(self.Zacc)
        self.mae = Sample('mae')
        self.adapter.add_data_item(self.mae)
        ## Add more samples below, if needed.

        # For events
        self.execution = Event('execution')
        self.adapter.add_data_item(self.execution)
        self.condition = Event('condition')
        self.adapter.add_data_item(self.condition)
        ## Add more events below, if needed.

        # MTConnect adapter availability
        self.avail = Event('avail')
        self.adapter.add_data_item(self.avail)

```

```

# Start MTConnect
self.adapter.start()
self.adapter.begin_gather()
self.avail.set_value("AVAILABLE")
self.adapter.complete_gather()

def adapter_stream(self, model, min_val, max_val, threshold):
    while True:
        try:
            x, y, z = measureData(acc,1000) # x=x-axis, y=y-axis, z-axis acceleration array
            input_feature = y
            input_feature_normalized = tensorNormalization(input_feature, min_val, max_val)
            result = predict(model, input_feature_normalized, threshold)
            x_rms = timeFeatures(x)[2] # calculate rms of x-axis
            y_rms = timeFeatures(y)[2] # calculate rms of y-axis
            z_rms = timeFeatures(z)[2] # calculate rms of z-axis
            mae = result[1] # mean absolute error (loss) of the model

            # if AFF is running:
            if x_rms > 0.4 and y_rms > 0.5:
                execution = 'ACTIVE'
                # if AFF is normal:
                if result[0]:
                    condition = 'NORMAL'
                else:
                    condition = 'ABNORMAL'
            else:
                execution = 'STOPPED'
                condition = 'UNAVAILABLE'

            now = datetime.datetime.now() # get current data time

            self.adapter.begin_gather() # start to collection

            self.Xacc.set_value(str(x_rms))
            self.Yacc.set_value(str(y_rms))
            self.Zacc.set_value(str(z_rms))
            self.execution.set_value(str(execution))
            self.condition.set_value(str(condition))
            self.mae.set_value(str(mae))

            self.adapter.complete_gather() # end of collection

            print("{} MTConnect data collection completed ... ".format(now)) # Printing out completed MT
            print("X-axis rms={0:.4f} m/s2, Y-axis rms={1:.4f} m/s2, Z-axis rms={2:.4f} m/s2".format(x_r,
            print("Execution={0}, condition={1}, loss={2:.4f}.\n".format(execution,condition,mae))

            # print("Power meter: Machine is now {}, {} W\n".format(ps,p1)) # Printing out Power meter m

        except KeyboardInterrupt: # To stop MTConnect adapter, Ctrl + c
            print("Stopping MTConnect...")
            self.adapter.stop() # Stop adapter thread
            sys.exit() # Terminate Python

## ===== MAIN =====
if __name__ == "__main__":
    model_path = '/home/pi/Documents/lab/lab9/models/best_model'
    model = keras.layers.TFSMLayer(model_path, call_endpoint="serving_default")

    threshold = 0.1198
    min_val = -4.707191999999999
    max_val = 4.3933792

    # start MTConnect Adapter
    adapter = MTConnectAdapter('0.0.0.0', 7878) # Args: host ip, port number
    adapter.adapter_stream(model, min_val, max_val, threshold) # start streaming data to MTConnect, args: mi

```

Task 3.3

Capture '/current' response from the MTConnect agent as Figure 11 and attach it to the report.

- Confirm if every data item works as expected by running the AFF.

L9_Colab2.ipynbME-597-IIoT/labMTConnect Devfrom data_item iraspberry pi.pytValueError: AtteGoogle GeminiChat

Not secure10.0.0.224:5000/currentSummarize

Homepage - PurdPurdue PortalJohn Blair Hooded...Cash & stock | Sto...ASUS E-ServiceOneCampus | Mos...Your Courses | Gra...Other favorites

- creationTime: 2026-04-05T21:42:24Z
- sender: robertclaud
- instanceId: 1775423448
- version: 1.8.0.3
- bufferSize: 131072
- nextSequence: 647
- firstSequence: 1
- lastSequence: 646

Device: ADXL345; UUID: 001

Device : ADXL345

Samples

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-04-05T21:42:24.036249Z	Acceleration		Xacc	a1	643	0.441078654202282
2026-04-05T21:42:24.036249Z	Acceleration		Yacc	a2	644	0.625493936967574
2026-04-05T21:42:24.036249Z	Acceleration		Zacc	a3	645	9.56937517863067
2026-04-05T21:42:24.036249Z	Loss		mae	error1	646	0.080336295

Events

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-04-05T21:10:48.122372Z	Availability			001	9	UNAVAILABLE
2026-04-05T21:10:48.121106Z	AssetChanged			adxl345_asset_chg	1	UNAVAILABLE
2026-04-05T21:10:48.12215Z	AssetRemoved			adxl345_asset_rem	6	UNAVAILABLE
2026-04-05T21:42:23.063313Z	Condition		condition	c1	642	NORMAL
2026-04-05T21:42:23.063313Z	Execution		execution	e1	641	ACTIVE

- creationTime: 2022-04-01T03:51:43Z - sender: lamm-UltraPoint - instanceId: 1648783879 - version: 1.8.0.2 - bufferSize: 131072 - nextSequence: 52 - firstSequence: 1 - lastSequence: 51						
Device: ADXL345; UUID: 001						
Device : ADXL345						
Samples						
Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2022-04-01T03:51:43.15467Z	Acceleration		Xacc	a1	48	0.822723114117054
2022-04-01T03:51:43.15467Z	Acceleration		Yacc	a2	49	0.858075038336765
2022-04-01T03:51:43.15467Z	Acceleration		Zacc	a3	50	9.67068542083701
2022-04-01T03:51:43.15467Z	Loss		mae	error1	51	0.032767966
Events						
Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2022-04-01T03:51:31.048541Z	Availability		avail	001	10	AVAILABLE
2022-04-01T03:31:19.500637Z	AssetChanged			adxl345_asset_chg	1	UNAVAILABLE
2022-04-01T03:31:19.501909Z	AssetRemoved			adxl345_asset_rem	6	UNAVAILABLE
2022-04-01T03:31:19.501816Z	State		condition	c1	5	UNAVAILABLE
2022-04-01T03:51:32.217381Z	Execution		execution	e1	14	STOPPED

Figure 11 MTConnect agent '/current' response with MTConnect adapter

Please continue to [Lab 9.4 here](#).